

ReBAC V2.2 — Dynamic Authorization DSL (ADSL Compiler)

Branch: *ReBAC-V2.2* | **Builds on:** *V2.1 Production Rule Engine*

The Problem V2.1 Left Unsolved

In V2.1, the permission schema is **hardcoded in TypeScript**:

```
const schema: RebacSchema = {
  patient: {
    view: { operator: "OR", rules: [{ relation: "doctor_of" }, {
      relation: "contains", permission: "view" }] }
  }
};
```

This is unacceptable in production because:

- **Changing a permission rule requires redeploying the server**
- **Non-engineers cannot read or write authorization rules**
- **Schema and runtime are tightly coupled** — a type error in rules crashes the engine

The solution: a **purpose-built Domain Specific Language (DSL)** for authorization, compiled at startup.

The Authorization DSL (ADSL)

```
# hospital.auth

resource patient {
  relation doctor_of : user
  relation assigned_nurse : user
```

```

relation contains : ward

permission view =
  doctor_of
  OR assigned_nurse
  OR contains -> view
}

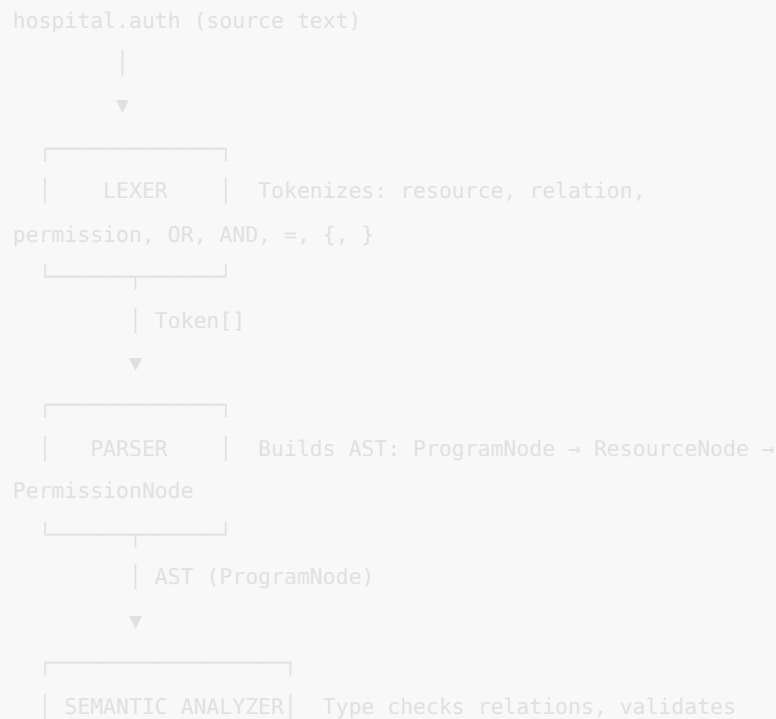
resource ward {
  relation assigned_nurse : user
  relation contains : department

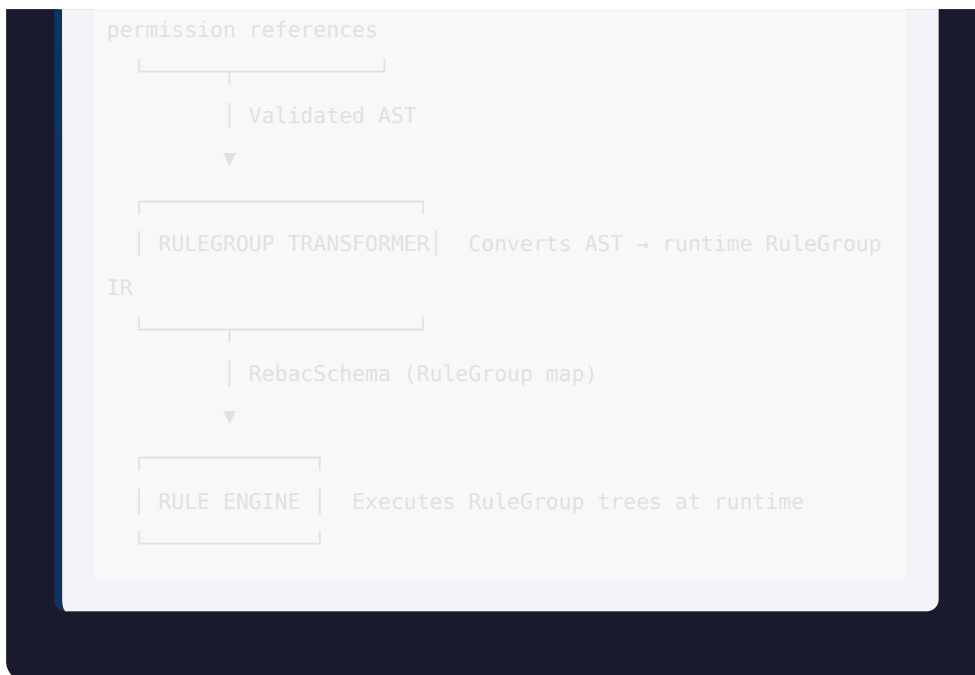
  permission view =
    assigned_nurse
    OR contains -> view
}

```

This is a **typed, declarative DSL** for expressing authorization rules in plain text, stored alongside the codebase as `.auth` files.

Compiler Architecture





AST Node Hierarchy

```
ProgramNode
├─ ResourceNode[]
│   └─ RelationDeclNode[] ("relation doctor_of : user")
│       └─ PermissionNode[]
│           └─ ExpressionNode (BinaryExpression | UnaryExpression | RelationNode)
```

Compiler Components

Component	File	Responsibility
Lexer	Lexer.ts	Tokenization — source text → Token[]
Token Types	TokenType.ts	Token enum (RESOURCE, RELATION, OR, ...)
Parser	Parser.ts	Token stream → AST (recursive descent)
AST Nodes	Nodes.ts	TypeScript interfaces for every node type

Semantic Analyzer	<code>SemanticAnalyzer.ts</code>	Type checking, undefined reference detection
Symbol Table	<code>SymbolTable.ts</code>	Tracks declared resources, relations, permissions
RuleGroup Transformer	<code>RuleGroupTransformer.ts</code>	AST → runtime IR
Compiler Facade	<code>CompilerFacade.ts</code>	Orchestrates all passes

Critical Review

✓ What V2.2 Gets Right

1. **Compiler architecture is correct.** Lexer → Parser → Semantic Analysis → IR is the classical pipeline used in production compilers (GCC, LLVM, TypeScript compiler).
2. **Separation of concerns.** The DSL decouples authorization policy from application code — matching how OPA (Open Policy Agent) uses Rego and AWS Cedar uses Cedar policy language.
3. **AST as the central data structure** is the correct design — every subsequent optimization, lint, and analysis operates on the AST.

⚠ What V2.2 Is Still Missing

Missing: Source maps When a runtime error occurs, there is no way to trace it back to the `.auth` file line/column. Production compilers always maintain source position metadata through all passes.

Missing: Error recovery The parser stops at the first error. Production compilers attempt to recover and report multiple errors simultaneously (like TypeScript's `--noEmitOnError false`).

Missing: Schema versioning No version field in `.auth` files. If the compiler changes, old files may parse incorrectly with no migration path.

Missing: Type system `relation doctor_of : user` declares a type, but the semantic analyzer doesn't enforce it at runtime. A nurse subject being assigned `doctor_of` on a patient would be accepted silently.

✗ Architectural Issue: RuleGroup as Final IR

The `RuleGroup` is both the IR and the runtime execution format. This conflates compilation and execution. A cleaner design uses:

- 1. **AST** — source-faithful representation
- 2. **HIR** (High-level IR) — semantically normalized, type-checked
- 3. **LIR / Execution Plan** — optimized, reordered for runtime

This separation is what V3.3 introduces with the Query Planner and ExecutionPlan.

Comparison with Industry Policy Languages

Language	Paradigm	Expressiveness	Runtime
ADSL (V2.2)	Declarative graph rules	Basic OR/AND/NOT, recursive	Custom engine
Rego (OPA)	Rego policy language	Datalog-like, general-purpose	Wasm/native
Cedar (AWS)	Declarative policies	PARC model, attribute-based	Rust engine
OpenFGA DSL	Authorization model	Zanzibar-faithful relations	Go engine
SpiceDB zed	Schema language	Typed, computed usersets	Go engine

ADSL is most similar to OpenFGA's DSL and SpiceDB's zed language — all three are Zanzibar-inspired, resource-centric DSLs.

Summary

V2.2 is the **most architecturally significant version** — it introduces the compiler that decouples authorization policy from application code. The `Lexer` → `Parser` → `SemanticAnalyzer` → `RuleGroupTransformer` pipeline is the correct classical compiler architecture. The remaining gaps (type enforcement, error recovery, source maps) are real production concerns, and the missing IR layering between AST and execution will be addressed in V3.3.